

Report conversation

We are building a content management system and need to model access control to be able to read, write, and/or manage specific pieces of content and also display contributor credits (bylines, credits, etc) on specific resources. The cms we are building is called Course Builder and is focused on researching, designing, producing, and delivering educational content online.

One challenge is that at the early stages of creating Course Builder we have two databases. One database is mysql and that is where the User is modeled. Specific pieces of content such as articles, video tips, and lessons are modeled and stored in Sanity.io.

here's the drizzle schema for the user:

```
export const users = mysqlTable(
  'user',
  {
    id: varchar('id', { length: 255 }).notNull().primaryKey(),
    name: varchar('name', { length: 255 }),
    role: mysqlEnum('role', ['user', 'admin']),
    email: varchar('email', { length: 255 }).notNull(),
    emailVerified: timestamp('emailVerified', {
      mode: 'date',
      fsp: 3,
    }),
    image: varchar('image', { length: 255 }),
    createdAt: timestamp('createdAt', {
      mode: 'date',
      fsp: 3,
    }).default(sql`CURRENT_TIMESTAMP(3)`),
  },
  (user) => ({
    emailIdx: index('email_idx').on(user.email),
    roleIdx: index('role_idx').on(user.role),
  })
)
```

```
    }},
  )
}
```

here's the Sanity schema for a Tip which is a specific type of resource that we publish that is part article part video:

```
import { MdOutlineLightbulb } from 'react-icons'
import { defineField } from 'sanity'
```

```
export default {
  name: 'tip',
  type: 'document',
  title: 'Tip',
  icon: MdOutlineLightbulb,
  fields: [
    defineField({
      name: 'title',
      title: 'Title',
      type: 'string',
      validation: (Rule) => Rule.required(),
    }),
    defineField({
      name: 'state',
      title: 'Current State',
      type: 'string',
      validation: (Rule) => Rule.required(),
      initialValue: 'draft',
      options: {
        list: [
          { title: 'draft', value: 'draft' },
          { title: 'published', value: 'published' },
        ],
      },
    }),
    defineField({
      name: 'visibility',
      title: 'Visibility',
      type: 'string',
      validation: (Rule) => Rule.required(),
    })
  ]
}
```

```

    initialValue: 'unlisted',
    options: {
      list: [
        { title: 'public', value: 'public' },
        { title: 'private', value: 'private' },
        { title: 'unlisted', value: 'unlisted' }
      ],
    },
  })),
  {
    name: 'resources',
    title: 'Resources',
    type: 'array',
    of: [{ type: 'reference', to: [{ type: 'resource' } ] },
    {
      name: 'body',
      title: 'Body',
      type: 'markdown',
    },
  ],
}

```

Inside of Course Builder we use CASL for role-based access control and assigning abilities and currently have only two roles, user and admin.

We need to consider how to model authorship and access control across these two databases with a preference for the mysql database as longer term we are trying to reduce and eliminate the use of Sanity.io and focus on building our own graph based cms.

With a focus on database schemas and modeling, make your best suggestion for this shape of system.

Given your requirements and setup, transitioning from using two databases (MySQL for users and Sanity.io for content) towards a unified MySQL-based system, particularly with an eye towards a graph-based CMS in the future, suggests a multi-phase approach. This

involves creating a flexible, extendable schema that not only handles current needs but also anticipates the eventual migration and integration of content from Sanity.io into MySQL.

Phase 1: Bridging the Current System

1. User Schema Adaptation:

- Your current `user` schema in MySQL is adequate for basic user management but lacks fields for handling authorship and contributions directly. Consider adding a `contributions` table that links users to their contributions across content types.

2. Access Control:

- Utilize CASL more dynamically to handle access control, by introducing a `permissions` table in MySQL. This table should map users and roles to specific permissions (read, write, manage) on content types or individual content items.

3. Content Metadata:

- Since your content currently resides in Sanity.io, create a `content_metadata` table in MySQL that stores content identifiers, types (e.g., `Tip`), and references to authors or contributors from the `user` table. This table acts as a bridge, allowing you to implement access control and authorship tracking.

Schema Suggestions

Users Table (Enhanced)

- No changes suggested currently, but consider adding fields for user profiles relevant to contributions, if not already present.

Contributions Table

sql

 Copy code

```
CREATE TABLE contributions (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  userId VARCHAR(255),  
  contentId VARCHAR(255),  
  contentType VARCHAR(50),  
  role ENUM('author', 'editor', 'contributor'),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (userId) REFERENCES user(id)
);
```

Permissions Table

sql

 Copy code

```
CREATE TABLE permissions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  userId VARCHAR(255),
  contentId VARCHAR(255),
  permissionType ENUM('read', 'write', 'manage'),
  FOREIGN KEY (userId) REFERENCES user(id)
);
```

Content Metadata Table

sql

 Copy code

```
CREATE TABLE content_metadata (
  contentId VARCHAR(255) PRIMARY KEY,
  contentType VARCHAR(50),
  state ENUM('draft', 'published'),
  visibility ENUM('public', 'private', 'unlisted'),
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  -- Additional fields for content-specific metadata can be added as needed
);
```

Phase 2: Towards a Graph-based CMS

- As you migrate content from Sanity.io to MySQL, evolve the `content_metadata` and `contributions` schemas into more graph-like structures, potentially leveraging MySQL 8.0's JSON functions or a dedicated graph database if necessary.
- Consider relationships not just between users and content, but also among content pieces themselves, to support a rich, interconnected content graph.
- Keep in mind the eventual goal of a graph-based system as you design your MySQL schemas, to make the transition smoother.

General Recommendations

- **Incremental Migration:** Start by synchronizing user and content metadata between MySQL and Sanity.io, ensuring that access control rules and contributions are accurately reflected in both systems.
- **Future-Proofing:** Design your MySQL schema with flexibility in mind, especially considering the eventual migration away from Sanity.io. Utilize abstract content identifiers and types to make it easier to adapt to new content models.
- **Testing and Validation:** Rigorously test the new access control and authorship tracking features before full migration, to ensure that the system behaves as expected and that data integrity is maintained.

This phased approach allows you to manage the complexity of operating across two databases while laying the groundwork for a unified, graph-based CMS.

go into more detail about the proposed `content_metadata` approach to bridging the two systems and potentially acting as a lever to migrate away from Sanity.io in the future

The `content_metadata` table acts as a central hub within your MySQL database for managing and referencing content that's currently stored in Sanity.io. The goal of this table is to create a schema that mirrors essential metadata about your content (such as articles, video tips, lessons, etc.), which allows for seamless integration between your user management system in MySQL and your content in Sanity.io. This approach not only facilitates current operational needs but also sets the stage for a smoother migration of content from Sanity.io to MySQL, eventually leading towards your goal of a graph-based CMS.

Detailed Schema Design for `content_metadata`

The `content_metadata` table should be designed to store enough information about each piece of content to manage access control, track authorship and contributions, and maintain basic content state and visibility settings. Here's a more detailed breakdown of the proposed schema:

sql

 Copy code

```
CREATE TABLE content_metadata (  
  contentId VARCHAR(255) PRIMARY KEY,  
  contentType VARCHAR(50),  
  title VARCHAR(255),  
  state ENUM('draft', 'published'),  
  visibility ENUM('public', 'private', 'unlisted'),  
  resources JSON, -- For storing references to other content or resources  
  body TEXT, -- Assuming some form of textual content; adjust type as needed  
  createdBy VARCHAR(255),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedBy VARCHAR(255),  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (createdBy) REFERENCES user(id),  
  FOREIGN KEY (updatedBy) REFERENCES user(id)  
);
```

Key Features and Functions

- **Content Identification:** The `contentId` serves as a unique identifier for each piece of content, facilitating a direct mapping between the records in this table and the actual content stored in Sanity.io.
- **Type and State Management:** Fields like `contentType` and `state` help categorize the content and manage its lifecycle from creation to publication.
- **Visibility Control:** The `visibility` field directly maps to your content's visibility settings in Sanity.io, enabling control over who can see what content.
- **Resource Linking:** Using a `JSON` field for `resources` allows for flexible linking to other content or external resources, a feature that's particularly useful when modeling complex relationships as you move towards a graph-based system.
- **Authorship and Audit Trail:** Including `createdBy`, `createdAt`, `updatedBy`, and `updatedAt` fields helps in tracking who contributed to the content and when, providing a clear audit trail.

Migration and Future Expansion

This schema not only bridges the current operational gap between your MySQL and Sanity.io setups but also prepares for the eventual full migration of content into MySQL. Here's how:

- **Content Syncing:** Initially, this table can be populated with metadata from Sanity.io content, keeping the systems in sync and allowing for dual operations.
- **Access Control Integration:** By linking `content_metadata` with the `permissions` and `contributions` tables, you establish a comprehensive access control and contribution tracking system that spans across all your content, regardless of where it's currently stored.
- **Migration Path:** When ready to migrate content fully into MySQL, the `content_metadata` table provides a clear blueprint for what content needs to be migrated, including its state, visibility, and relational links to other content or resources.

Leveraging for a Graph-based CMS

As you progress towards a graph-based CMS, the `content_metadata` table's structure—particularly the use of `JSON` for resources and the detailed tracking of content relationships—lays the groundwork for more complex content graph modeling. This approach facilitates the transition to a system where content is not just stored but is deeply interconnected, supporting advanced content discovery, recommendation, and personalization features.

In summary, the `content_metadata` approach is both a practical solution for immediate needs and a strategic foundation for future growth towards a more sophisticated, graph-based content management system.

list the primary drawbacks to the `content_metadata` approach and suggest practical alternatives that we should consider with Course Builder

The `content_metadata` approach, while offering a strategic bridge between your current MySQL and Sanity.io setups and facilitating migration to a graph-based CMS, comes with several drawbacks that need consideration. Understanding these limitations can help in exploring practical alternatives for Course Builder.

Primary Drawbacks

1. **Complex Syncing Logic:** Maintaining synchronization between MySQL's `content_metadata` and the actual content in Sanity.io requires complex logic, especially as content gets updated, deleted, or added in either system.
2. **Redundancy and Data Drift:** The risk of data redundancy and drift increases as you duplicate content metadata into MySQL. Keeping both systems in sync without discrepancies poses a challenge.
3. **Performance Overheads:** The additional layer of metadata management can introduce performance overheads, particularly as the volume of content grows and the frequency of access control checks increases.
4. **Migration Complexity:** While the `content_metadata` table is designed to facilitate migration, the actual process of moving content from Sanity.io to MySQL and ensuring all relationships and references are correctly mapped can be complex and error-prone.
5. **Development and Maintenance Costs:** Building and maintaining the sync mechanisms, as well as the additional access control logic on top of two separate systems, requires significant development effort and ongoing maintenance.

Practical Alternatives

Given these drawbacks, here are some alternatives and strategies to consider:

1. **Direct Integration via APIs:** Instead of syncing content metadata into MySQL, consider using Sanity.io's API directly from Course Builder to manage and access content. This reduces redundancy and sync complexity but requires robust API rate limiting and caching strategies to ensure performance.
2. **Event-Driven Synchronization:** Implement an event-driven architecture where changes in Sanity.io publish events (via webhooks or a similar mechanism) that trigger updates in MySQL. This approach can reduce the lag and complexity in keeping systems in sync but requires a solid event handling and error recovery mechanism.
3. **Hybrid Content Storage Solution:** Explore a hybrid approach where MySQL is used for structured content management and access control, while Sanity.io is reserved for rich content editing and versioning. This can reduce the burden on MySQL while leveraging the strengths of Sanity.io.
4. **Gradual Migration with Feature Flags:** If moving away from Sanity.io is a long-term goal, consider a gradual migration strategy enabled by feature flags. This allows you

to slowly transfer content and functionality to the MySQL-based system on a case-by-case basis, reducing the risks of a big-bang migration.

5. **Fully Managed Database Services:** Consider using fully managed graph database services that offer sophisticated access control and relationship management features out of the box. This could significantly reduce development overhead and provide a more scalable foundation for your graph-based CMS ambitions.
6. **Serverless Functions for Sync Logic:** Use serverless functions to handle the synchronization between Sanity.io and MySQL. This can isolate the sync logic, making it more manageable and scalable, and reduce the load on your primary application servers.

Each of these alternatives has its own set of trade-offs in terms of complexity, performance, and future scalability. The choice depends on your specific priorities, such as the importance of reducing dependency on Sanity.io, the need for a unified access control model, and the timeline for migrating to a graph-based CMS. Balancing these factors against the drawbacks of the `content_metadata` approach will guide you towards the most practical solution for Course Builder.

We have implemented an event-driven system that is able to execute serverless functions within the context of our application. Knowing that, how would this affect the proposed architecture for Course Builder?

Given the implementation of an event-driven system capable of executing serverless functions within the context of Course Builder, this introduces a dynamic and flexible architecture that can significantly enhance your approach to managing access control, authorship, and the gradual migration away from Sanity.io. Here's how it can affect and potentially improve the proposed architecture:

Improved Synchronization

- **Real-time Updates:** Serverless functions can be triggered by events in both your MySQL and Sanity.io databases, ensuring real-time updates and synchronization between the content metadata in MySQL and the actual content in Sanity.io. This minimizes the risk of data drift and redundancy.

- **Scalability:** Event-driven architectures are inherently scalable. As your content and user base grow, serverless functions can automatically scale to handle increased loads, making your system more resilient and responsive.

Enhanced Access Control Logic

- **Dynamic Access Control:** Serverless functions can dynamically enforce access control rules by interfacing with both databases. This allows for more complex and context-sensitive access control decisions, beyond what static roles and permissions allow.
- **Customizable Authorization:** With the flexibility of serverless functions, you can implement custom logic for specific scenarios, such as temporary access, role escalation under certain conditions, or integrating third-party authentication services.

Streamlined Migration Strategy

- **Incremental Migration:** Serverless functions can facilitate an incremental migration process from Sanity.io to MySQL (and eventually to your graph-based CMS) by handling data transformation, validation, and transfer on an as-needed basis. This reduces the migration's impact on your live system.
- **Validation and Error Handling:** During the migration, serverless functions can also perform data validation, error logging, and rollback procedures, ensuring data integrity and minimizing disruption.

Potential for Advanced Features

- **Personalization and Analytics:** The flexibility and scalability offered by serverless functions, combined with event-driven architecture, open up possibilities for implementing advanced features such as content personalization, real-time analytics, and user behavior tracking without overloading your primary databases.
- **Automated Content Management Workflows:** You can automate complex content management workflows, such as content review and approval processes, notifications for content updates, and automated content tagging or categorization.

Considerations

- **Complexity Management:** While serverless functions add flexibility and power, managing a large number of these functions, especially their triggers, inputs, and outputs, can become complex. Adopting best practices for serverless architecture,

such as maintaining clear documentation, modularizing functions, and implementing robust monitoring and logging, is essential.

- **Cost Control:** Although serverless functions can be cost-effective at smaller scales or for variable workloads, it's important to monitor and optimize usage to prevent cost overruns, especially as the number of functions and their execution frequency increases.

Incorporating serverless functions into your event-driven architecture for Course Builder not only addresses the immediate needs around synchronization, access control, and migration but also lays a flexible foundation for future expansion and innovation.

```
CREATE TABLE content_metadata (
  contentId VARCHAR(255) PRIMARY KEY,
  contentType VARCHAR(50),
  title VARCHAR(255),
  state ENUM('draft', 'published'),
  visibility ENUM('public', 'private', 'unpublished'),
  resources JSON, -- For storing reference data
  body TEXT, -- Assuming some form of text content
  createdBy VARCHAR(255),
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedBy VARCHAR(255),
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (createdBy) REFERENCES users (id),
  FOREIGN KEY (updatedBy) REFERENCES users (id)
);
```

here's the current suggestion for content_metadata. update it based on your best understanding of Course Builder and give me an example of what the JSON stored in resources will look like. contentType is probably its own table so they can be user defined with schemas

Given your requirements and the dynamic nature of Course Builder, separating contentType into its own table and making resources flexible for user-defined

schemas are strategic moves. This approach enhances modularity, scalability, and customizability of your content management system. Here's an updated version of the `content_metadata` table and a new `content_type` table, along with an example JSON for resources .

Updated `content_metadata` Table

sql

 Copy code

```
CREATE TABLE content_metadata (  
  contentId VARCHAR(255) PRIMARY KEY,  
  contentTypeId INT,  
  title VARCHAR(255),  
  state ENUM('draft', 'published'),  
  visibility ENUM('public', 'private', 'unlisted'),  
  resources JSON, -- For storing references to other content or resources  
  body TEXT, -- Assuming some form of textual content; adjust type as needed  
  createdBy VARCHAR(255),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedBy VARCHAR(255),  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (createdBy) REFERENCES user(id),  
  FOREIGN KEY (updatedBy) REFERENCES user(id),  
  FOREIGN KEY (contentTypeId) REFERENCES content_type(id)  
);
```

New `content_type` Table

This table allows users to define and manage different types of content, including custom schemas for each type.

sql

 Copy code

```
CREATE TABLE content_type (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  typeName VARCHAR(50),  
  schemaDefinition JSON, -- Stores user-defined schema for this content type  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);
```

Example resources JSON

The `resources` field in the `content_metadata` table can store complex and varied references to other content or resources. The structure of this JSON can be designed to fit the specific needs of different content types, offering immense flexibility. Here's an example JSON that references video and article resources:

json

 Copy code

```
{  
  "videos": [  
    {  
      "videoId": "v123",  
      "title": "Introduction to Course Builder",  
      "url": "https://example.com/videos/intro"  
    }  
  ],  
  "articles": [  
    {  
      "articleId": "a456",  
      "title": "Getting Started with Course Builder",  
      "url": "https://example.com/articles/getting-started"  
    }  
  ],  
  "externalResources": [  
    {  
      "name": "Course Builder GitHub Repository",  
      "url": "https://github.com/coursebuilder/project"  
    }  
  ]  
}
```

This example illustrates how you can store references to various types of resources, including both internal content (videos, articles) and external links. This structure is just a starting point; you can adjust the schema to include any type of resource relevant to your courses, such as quizzes, downloadable files, or links to external tools.

The flexibility of JSON allows for dynamically adjusting the content relationships as your system evolves, especially as you transition more functionality and content definitions into your own graph-based CMS.

show an example of defining a serializable schema using
Zod in TypeScript for schemaDefinition

Zod is a TypeScript-first schema declaration and validation library that allows you to build complex validations for your data structures. When defining a serializable schema for `schemaDefinition` in your `content_type` table, you would define the schema structure in TypeScript using Zod, then serialize it into a JSON string to store in the database. This JSON string can later be deserialized back into a Zod schema for runtime validation.

Here's an example of how you might define a serializable schema using Zod for a content type, such as a "Tip" document, and then serialize it:

Defining the Schema with Zod

typescript

 Copy code

```
import { z } from 'zod';

// Define a Zod schema for a "Tip" content type
const TipSchema = z.object({
  title: z.string().min(1, "Title is required"),
  state: z.enum(['draft', 'published']),
  visibility: z.enum(['public', 'private', 'unlisted']),
  resources: z.array(z.object({
    type: z.string(),
    id: z.string(),
    title: z.string().optional(),
    url: z.string().url().optional()
  })),
  body: z.string(),
});
```

```
// Example of how you might serialize this schema to store it in your data  
const serializedSchemaDefinition = JSON.stringify(TipSchema.toJSON());  
  
console.log(serializedSchemaDefinition);
```

Serializing the Schema

The `TipSchema.toJSON()` method serializes the schema into a JSON format that Zod can understand. This JSON representation can be stored in your `content_type.schemaDefinition` column. When you need to validate data against this schema, you would deserialize this JSON back into a Zod schema.

Deserializing and Using the Schema

To use the stored schema for validation, you'd first retrieve the serialized schema from your database, then parse it back into a Zod schema object. However, Zod doesn't directly support deserialization from JSON to a Zod schema object as of my last update. As a workaround, you'd typically use the serialized form for documentation or code generation purposes, and maintain the Zod schema in your codebase for actual validation.

For a true dynamic schema system, you might need to build or use a custom solution to interpret and validate against the JSON schema definitions at runtime, or use the serialized schemas as a basis for generating Zod schemas as part of your build process.

Note

This approach with Zod shows the conceptual process of serializing and potentially deserializing schema definitions. Due to Zod's nature and TypeScript's compile-time type system, dynamically creating Zod schemas from a database at runtime involves additional steps not natively supported by Zod, such as using the serialized schema for documentation or leveraging it in a build step to generate TypeScript types and Zod validators.

more detail on this: "leveraging it in a build step to generate TypeScript types and Zod validators."

Leveraging serialized schema definitions from a database in a build step to generate TypeScript types and Zod validators involves a few steps. This process can enable dynamic schema validation in your application based on user-defined schemas stored in your database. Here's a more detailed breakdown:

Step 1: Schema Design and Storage

First, design your content schemas using Zod in your development environment. Serialize these schemas to JSON and store them in your database, as outlined previously. This step is done manually by developers as they design the content models for the application.

Step 2: Retrieving Serialized Schemas

At build time or during a deployment process, retrieve the serialized schema definitions from your database. This could be done through a script that queries your database for all the schema definitions stored in the `content_type` table.

Step 3: Generating Zod Validators and TypeScript Types

Once you have the serialized schemas, you will use a custom script or tool to convert them back into Zod schema objects and TypeScript types. Since Zod does not support direct deserialization from JSON to Zod schemas, this step requires a bit of creativity:

- **Custom Deserialization Logic:** You might develop a function or script that interprets the JSON schema and dynamically creates Zod schema objects based on that definition. This script maps the JSON structure back to Zod's API calls.
- **TypeScript Code Generation:** For generating TypeScript types, you could use Zod's ability to infer TypeScript types from Zod schemas. Using the dynamically created Zod schemas, you can generate TypeScript type declarations. This might involve using the TypeScript Compiler API to emit type definitions programmatically or templating tools to generate TypeScript code files.

Step 4: Integrating Generated Code into Your Application

The final step is to integrate the generated Zod validators and TypeScript types into your application. This could mean:

- Automatically generating files containing the Zod schemas and TypeScript types before your application builds. These files are then imported into your application codebase where needed.

- Dynamically importing these validators and types into your application at runtime, if your application structure and hosting environment allow for such a dynamic code execution model.

Practical Example

Imagine you have a content type `tip` in your database with a serialized schema. During your build process, a script runs, fetching the serialized schema, generating a Zod schema object, and then using Zod to generate TypeScript types. The output might be a file like `TipSchema.ts` with content something like this:

typescript

 Copy code

```
// Generated by your build script
import { z } from 'zod';

export const TipSchema = z.object({
  title: z.string().min(1, "Title is required"),
  // The rest of your schema...
});

export type Tip = z.infer<typeof TipSchema>;
```

This file is then used in your application to validate `tip` content types and to provide TypeScript types for better developer experience and type safety.

Challenges and Considerations

- **Complexity:** The process adds complexity to your build and deployment pipeline.
- **Custom Scripting:** You'll need to maintain custom scripts for schema conversion and type generation.
- **Runtime Performance:** Consider the implications of dynamic schema loading and validation on your application's performance and start-up time.
- **Versioning and Updates:** Managing updates to schemas and ensuring your generated code stays in sync with your database schemas require careful planning.

This approach allows for a high degree of flexibility and enables dynamic schema validation based on user-defined models, but it's essential to weigh these benefits against the added complexity and maintenance overhead.

show me the drizzle mysql schemas that summarize the above conversation for users, contributions, permissions, and content metadata. keep the explaining to a minimum and let the sql speak for you

sql

 Copy code

```
-- Users Table
CREATE TABLE users (
  id VARCHAR(255) PRIMARY KEY,
  name VARCHAR(255),
  role ENUM('user', 'admin') DEFAULT 'user',
  email VARCHAR(255) NOT NULL,
  emailVerified TIMESTAMP(3),
  image VARCHAR(255),
  createdAt TIMESTAMP(3) DEFAULT CURRENT_TIMESTAMP(3),
  INDEX email_idx (email),
  INDEX role_idx (role)
);

-- Content Types Table
CREATE TABLE content_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50),
  schemaDefinition JSON NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Content Metadata Table
CREATE TABLE content_metadata (
  contentId VARCHAR(255) PRIMARY KEY,
  typeId INT,
  title VARCHAR(255),
  state ENUM('draft', 'published'),
  visibility ENUM('public', 'private', 'unlisted'),
  resources JSON, -- Storing references to other content or resources
```

```
body TEXT,  
createdBy VARCHAR(255),  
createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updatedBy VARCHAR(255),  
updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
FOREIGN KEY (createdBy) REFERENCES users(id),  
FOREIGN KEY (updatedBy) REFERENCES users(id),  
FOREIGN KEY (typeId) REFERENCES content_types(id)  
);
```

-- Contributions Table

```
CREATE TABLE contributions (  
  contentId VARCHAR(255),  
  userId VARCHAR(255),  
  contributionType ENUM('author', 'editor', 'reviewer'),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (contentId, userId, contributionType),  
  FOREIGN KEY (contentId) REFERENCES content_metadata(contentId),  
  FOREIGN KEY (userId) REFERENCES users(id)  
);
```

-- Permissions Table

```
CREATE TABLE permissions (  
  userId VARCHAR(255),  
  contentId VARCHAR(255),  
  permissionType ENUM('read', 'write', 'manage'),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (userId, contentId, permissionType),  
  FOREIGN KEY (userId) REFERENCES users(id),  
  FOREIGN KEY (contentId) REFERENCES content_metadata(contentId)  
);
```

rewrite it with contributionType and
permissionType as independent tables and user-
defined

-- Users Table

```
CREATE TABLE users (  
  id VARCHAR(255) PRIMARY KEY,  
  name VARCHAR(255),  
  role ENUM('user', 'admin') DEFAULT 'user',  
  email VARCHAR(255) NOT NULL,  
  emailVerified TIMESTAMP(3),  
  image VARCHAR(255),  
  createdAt TIMESTAMP(3) DEFAULT CURRENT_TIMESTAMP(3),  
  INDEX email_idx (email),  
  INDEX role_idx (role)  
);
```

-- Content Types Table

```
CREATE TABLE content_types (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50),  
  schemaDefinition JSON NOT NULL,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);
```

-- Contribution Types Table

```
CREATE TABLE contribution_types (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) UNIQUE NOT NULL,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);
```

-- Permission Types Table

```
CREATE TABLE permission_types (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) UNIQUE NOT NULL,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);
```

-- Content Metadata Table

```
CREATE TABLE content_metadata (  
  contentId VARCHAR(255) PRIMARY KEY,  
  typeId INT,  
  title VARCHAR(255),  
  state ENUM('draft', 'published'),  
  visibility ENUM('public', 'private', 'unlisted'),  
  resources JSON,  
  body TEXT,  
  createdBy VARCHAR(255),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedBy VARCHAR(255),  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (createdBy) REFERENCES users(id),  
  FOREIGN KEY (updatedBy) REFERENCES users(id),  
  FOREIGN KEY (typeId) REFERENCES content_types(id)  
);
```

-- Contributions Table

```
CREATE TABLE contributions (  
  contentId VARCHAR(255),  
  userId VARCHAR(255),  
  contributionTypeId INT,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (contentId, userId, contributionTypeId),  
  FOREIGN KEY (contentId) REFERENCES content_metadata(contentId),  
  FOREIGN KEY (userId) REFERENCES users(id),  
  FOREIGN KEY (contributionTypeId) REFERENCES contribution_types(id)  
);
```

-- Permissions Table

```
CREATE TABLE permissions (  
  userId VARCHAR(255),  
  contentId VARCHAR(255),  
  permissionTypeId INT,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (userId, contentId, permissionTypeId),  
  FOREIGN KEY (userId) REFERENCES users(id),  
  FOREIGN KEY (contentId) REFERENCES content_metadata(contentId),
```

```
FOREIGN KEY (permissionTypeId) REFERENCES permission_types(id)
);
```

rewrite again with role on user as a user defined table

sql

 Copy code

-- Users Table

```
CREATE TABLE users (
  id VARCHAR(255) PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255) NOT NULL,
  emailVerified TIMESTAMP(3),
  image VARCHAR(255),
  createdAt TIMESTAMP(3) DEFAULT CURRENT_TIMESTAMP(3),
  INDEX email_idx (email)
);
```

-- Roles Table

```
CREATE TABLE roles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
);
```

-- User Roles Junction Table

```
CREATE TABLE user_roles (
  userId VARCHAR(255),
  roleId INT,
  PRIMARY KEY (userId, roleId),
  FOREIGN KEY (userId) REFERENCES users(id),
  FOREIGN KEY (roleId) REFERENCES roles(id)
);
```

-- Content Types Table

```
CREATE TABLE content_types (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50),  
  schemaDefinition JSON NOT NULL,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);  
  
-- Contribution Types Table  
CREATE TABLE contribution_types (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) UNIQUE NOT NULL,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);  
  
-- Permission Types Table  
CREATE TABLE permission_types (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) UNIQUE NOT NULL,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
);  
  
-- Content Metadata Table  
CREATE TABLE content_metadata (  
  contentId VARCHAR(255) PRIMARY KEY,  
  typeId INT,  
  title VARCHAR(255),  
  state ENUM('draft', 'published'),  
  visibility ENUM('public', 'private', 'unlisted'),  
  resources JSON,  
  body TEXT,  
  createdBy VARCHAR(255),  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedBy VARCHAR(255),  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (createdBy) REFERENCES users(id),  
  FOREIGN KEY (updatedBy) REFERENCES users(id),  
  FOREIGN KEY (typeId) REFERENCES content_types(id)
```



```
);
```

```
-- Contributions Table
```

```
CREATE TABLE contributions (  
  contentId VARCHAR(255),  
  userId VARCHAR(255),  
  contributionTypeId INT,  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (contentId, userId, contributionTypeId),  
  FOREIGN KEY (contentId) REFERENCES content_metadata(contentId),  
  FOREIGN KEY (userId) REFERENCES users(id),  
  FOREIGN KEY (contributionTypeId) REFERENCES contribution_types(id)  
);
```